

deeplearning.ai

One hidden layer Neural Network

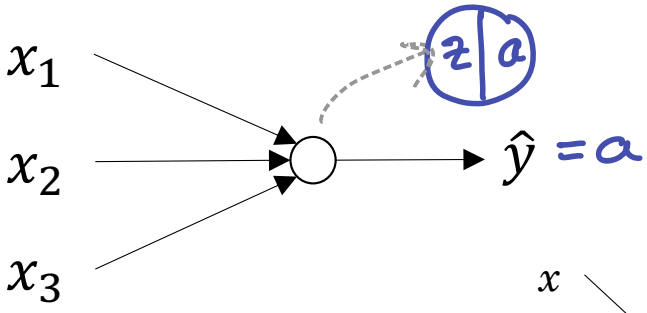
Neural Networks Overview

Learn to build a neural network with one hidden layer, using forward propagation and backpropagation.

Learning Objectives

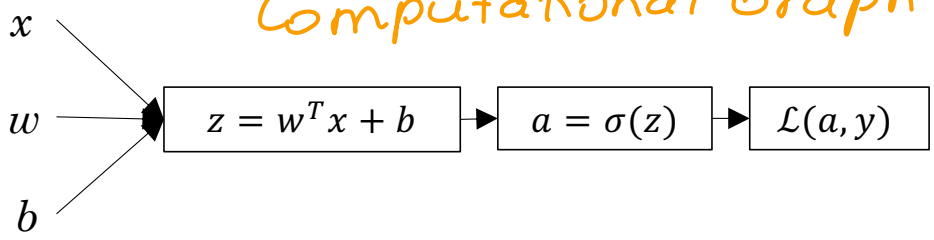
- Describe hidden units and hidden layers
- Use units with a non-linear activation function, such as tanh
- Implement forward and backward propagation
- Apply random initialization to your neural network
- Increase fluency in Deep Learning notations and Neural Network Representations

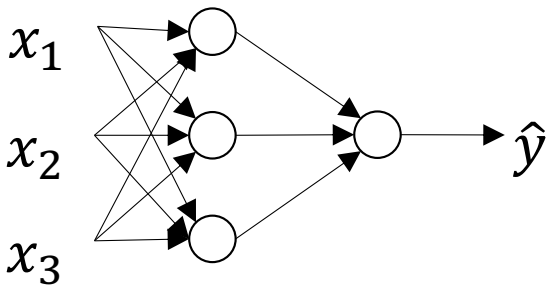
What is a Neural Network?



LOGISTIC REGRESSION

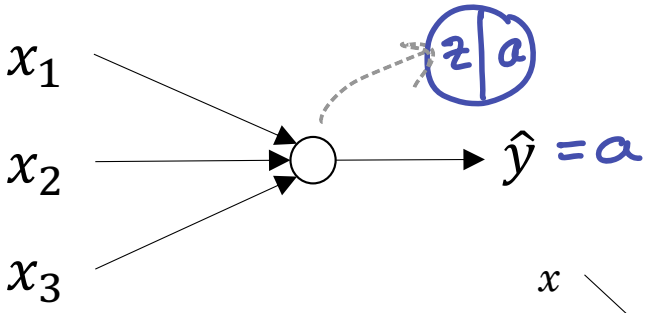
Computational Graph:





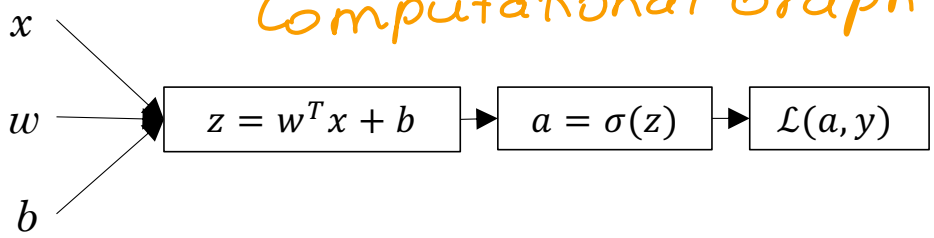
NEURAL NETWORK

What is a Neural Network?



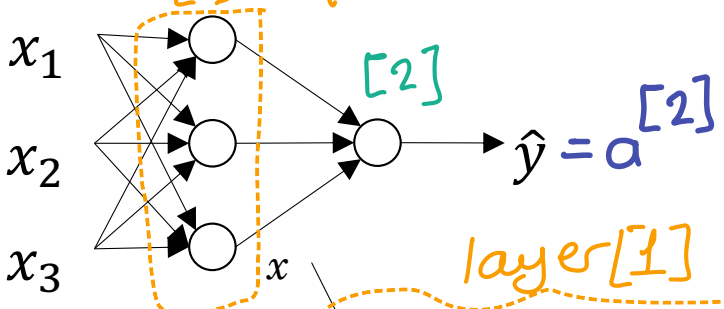
LOGISTIC REGRESSION

Computational Graph: z followed by a calculation



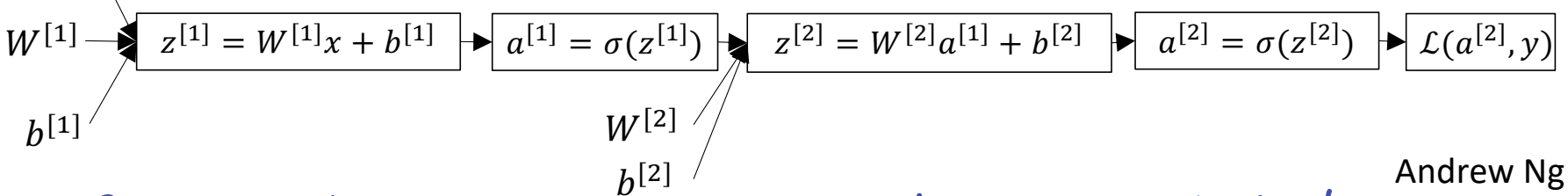
[1]: layer

NEURAL NETWORK



layer [1]

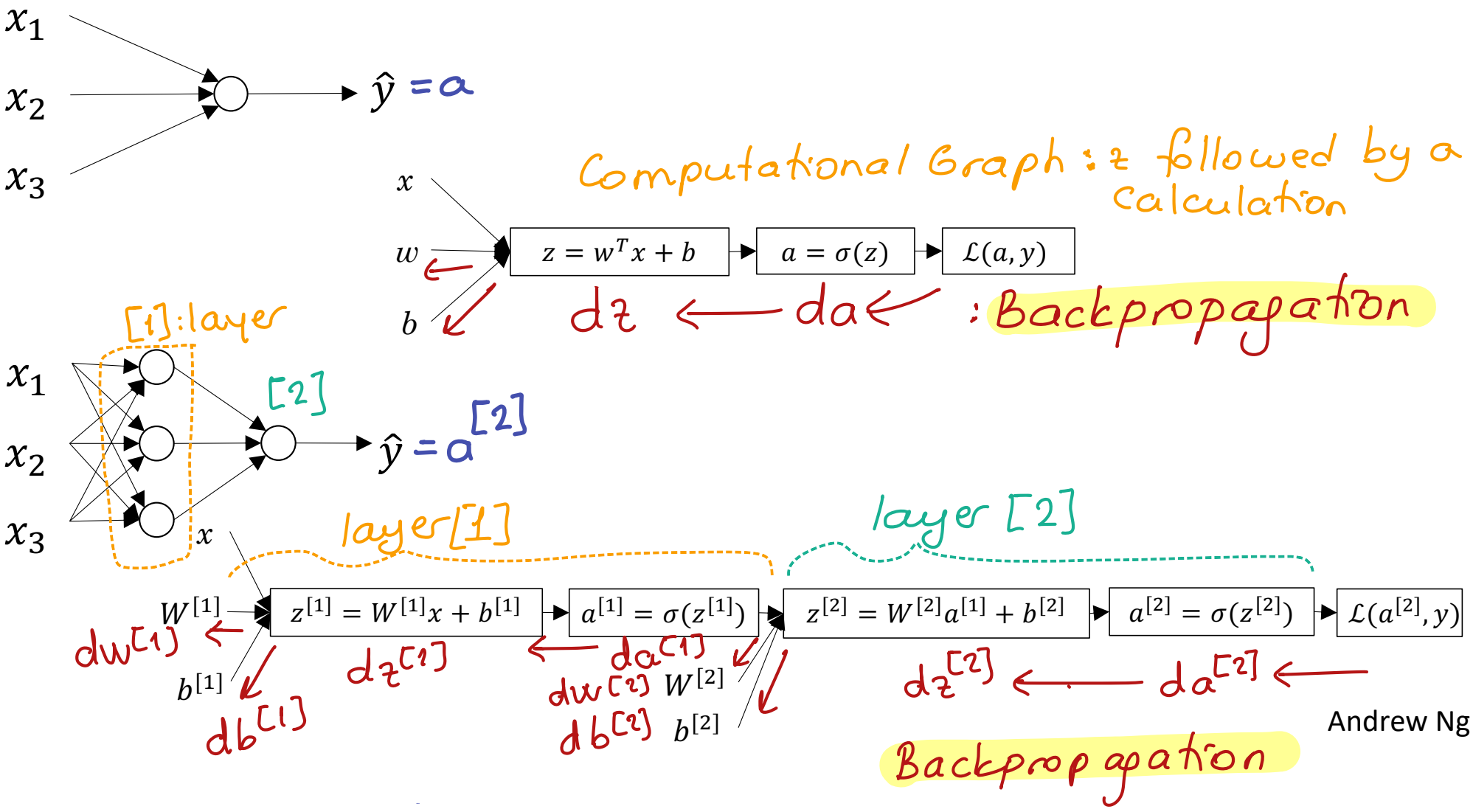
layer [2]



Andrew Ng

- NN: z followed by a calculation is done multiple times.

What is a Neural Network?



Andrew Ng

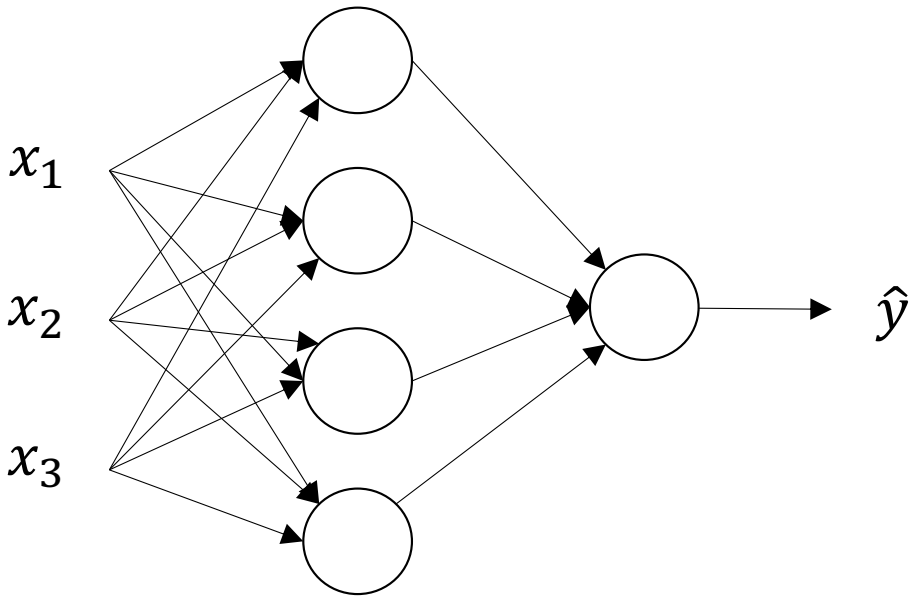


deeplearning.ai

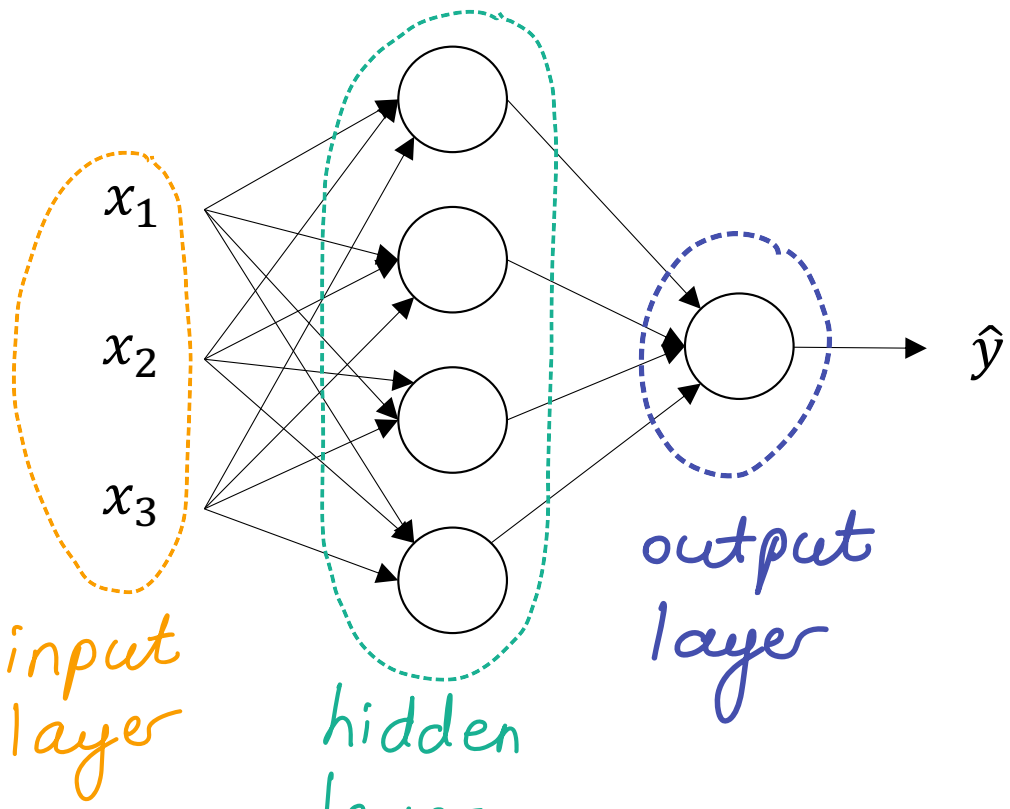
One hidden layer Neural Network

Neural Network Representation

Neural Network Representation



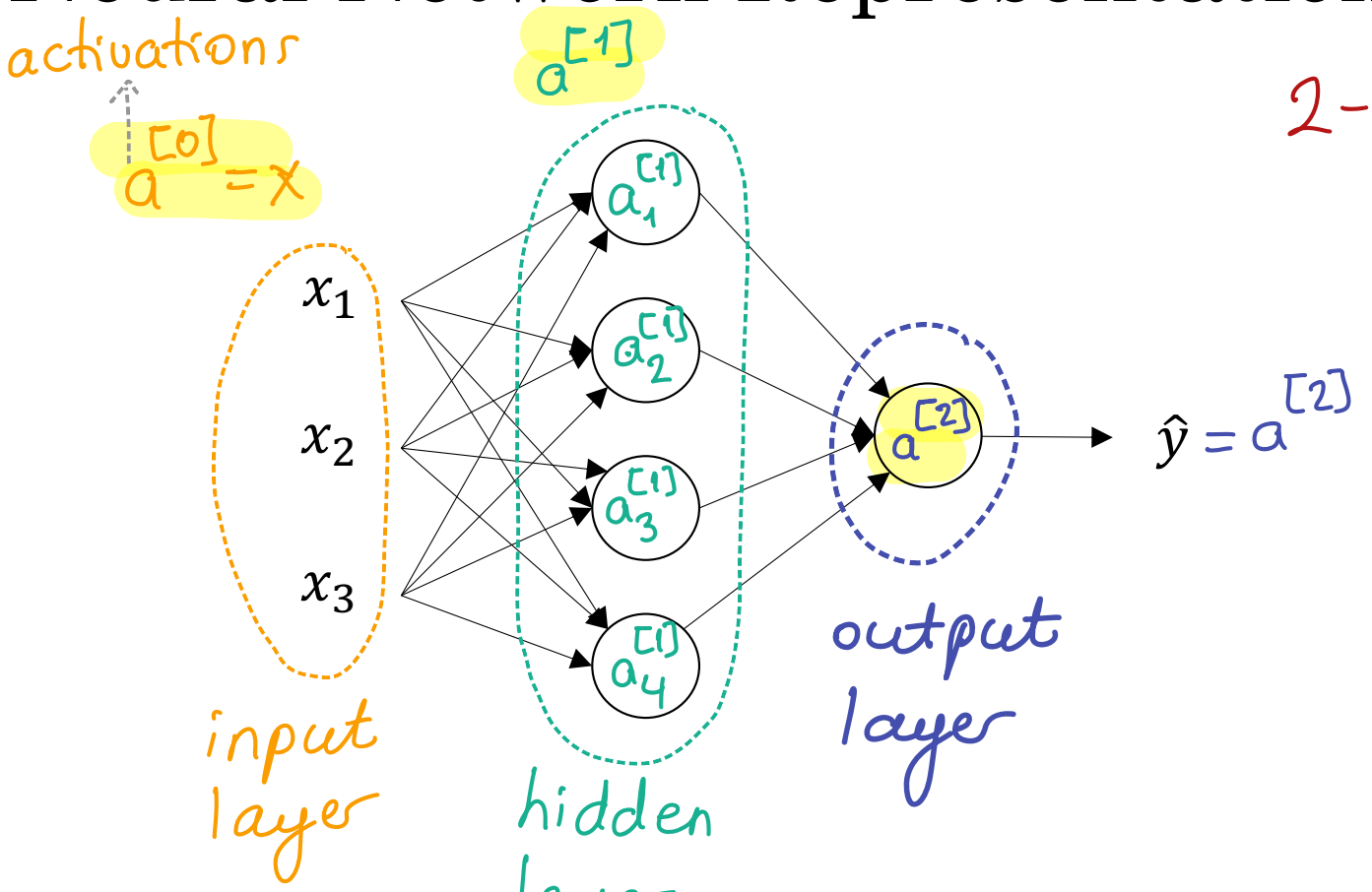
Neural Network Representation : Layers



↳ In a traing set we don't see the output values of this layer. (they're hidden)

Andrew Ng

Neural Network Representation: activations



Andrew Ng

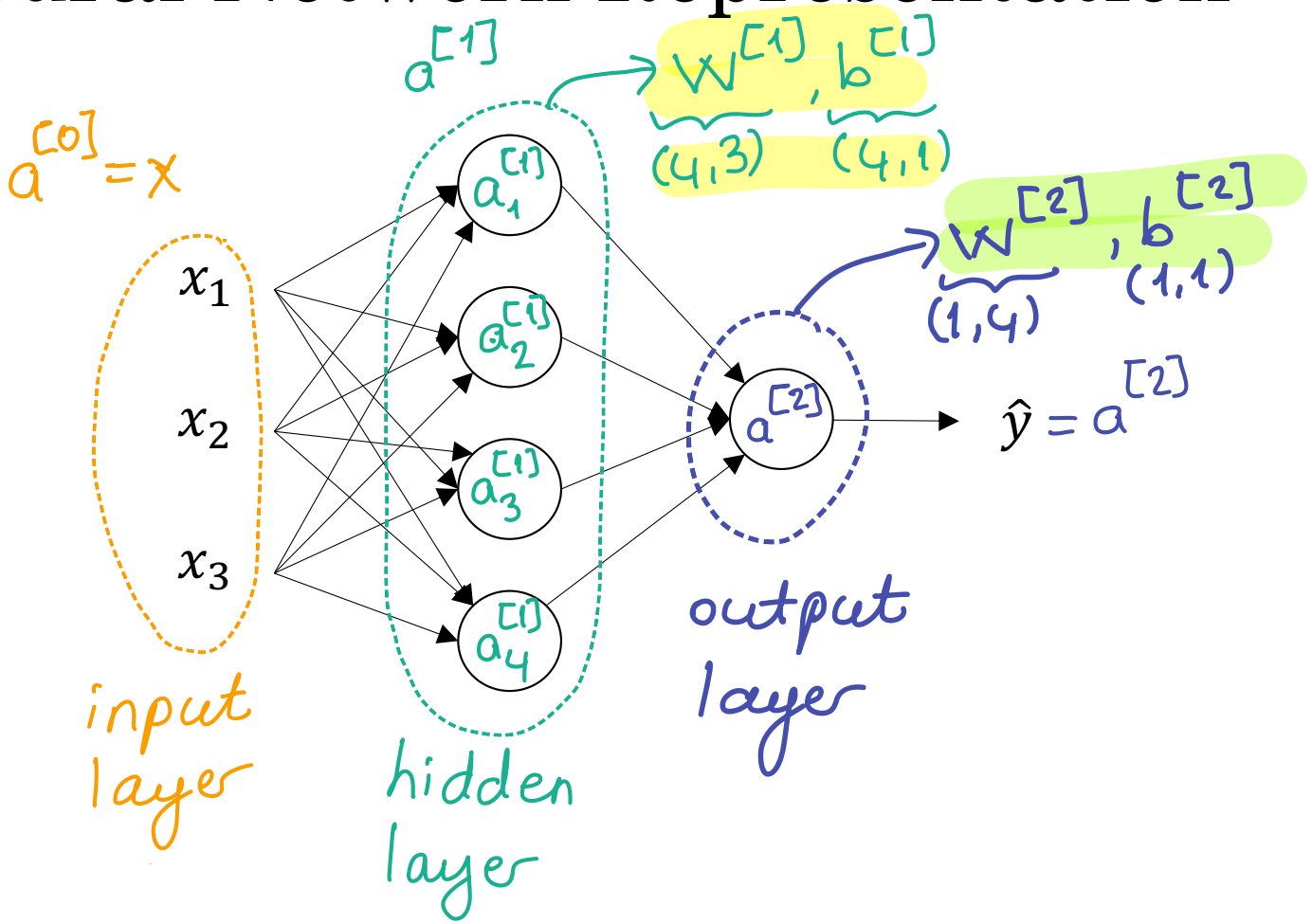
$a^{[0]}$: activations of input layer

$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix}_{4 \times 1}$$

there are 4 hidden units in this layer.

2-layer-NN : we don't count the input layer, by notation.
Input layer is layer 0.

Neural Network Representation: Parameters



Andrew Ng



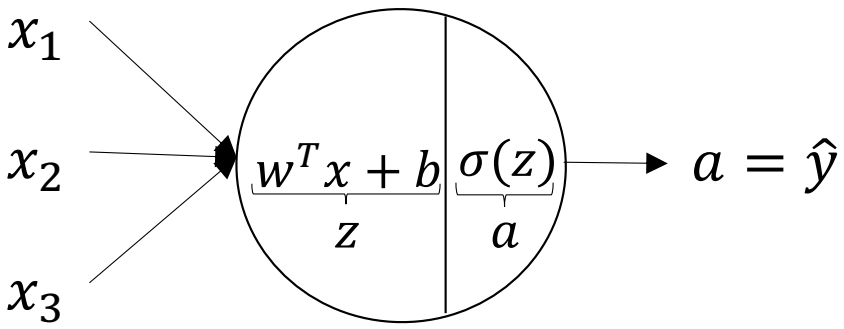
deeplearning.ai

One hidden layer Neural Network

Computing a Neural Network's Output

Neural Network Representation

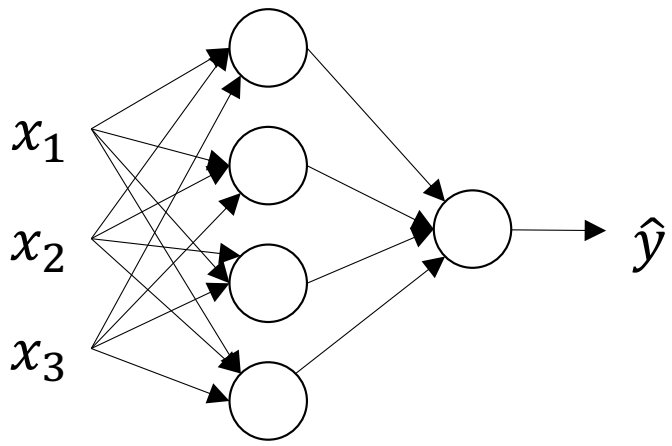
LOGISTIC REGRESSION



$$z = w^T x + b$$

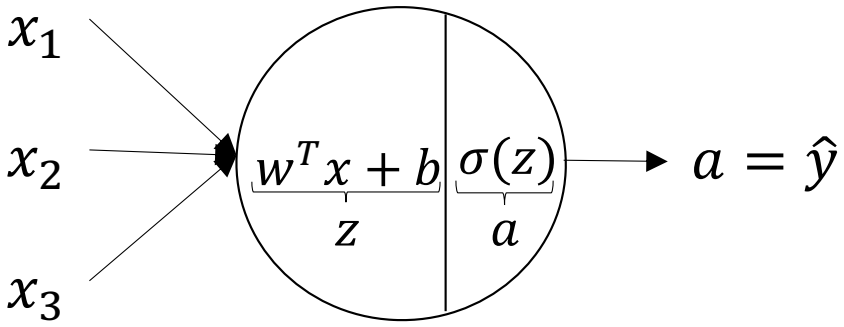
$$a = \sigma(z)$$

NEURAL NETWORK



- Repeats logistic regression computations many times

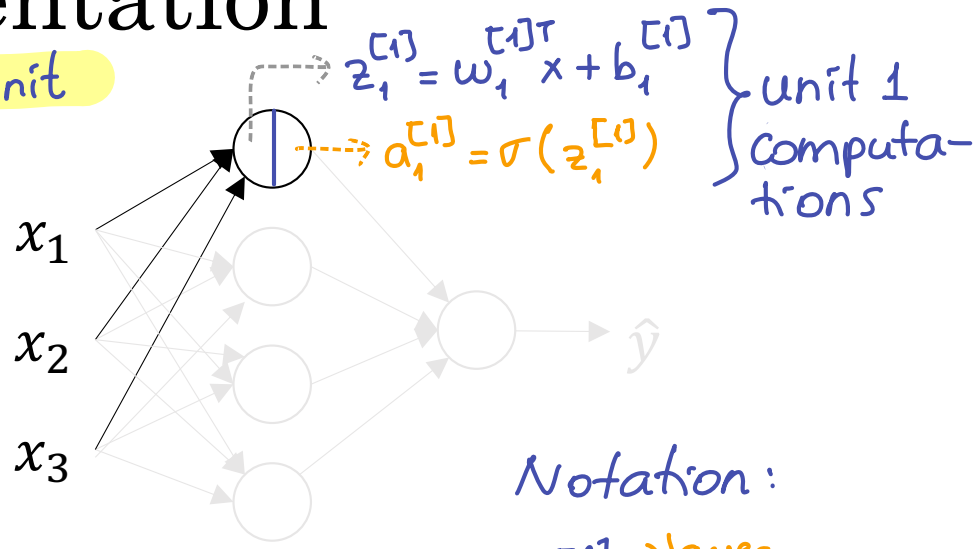
Neural Network Representation



$$z = w^T x + b$$

$$a = \sigma(z)$$

First Unit



Notation:

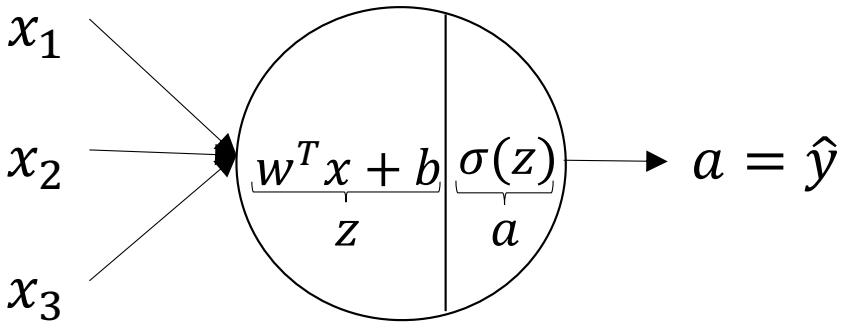
$a_i^{[l]}$ → layer

→ unit

$w_1^{[1]T} = (w_1^{[1]})^T$ → transpose

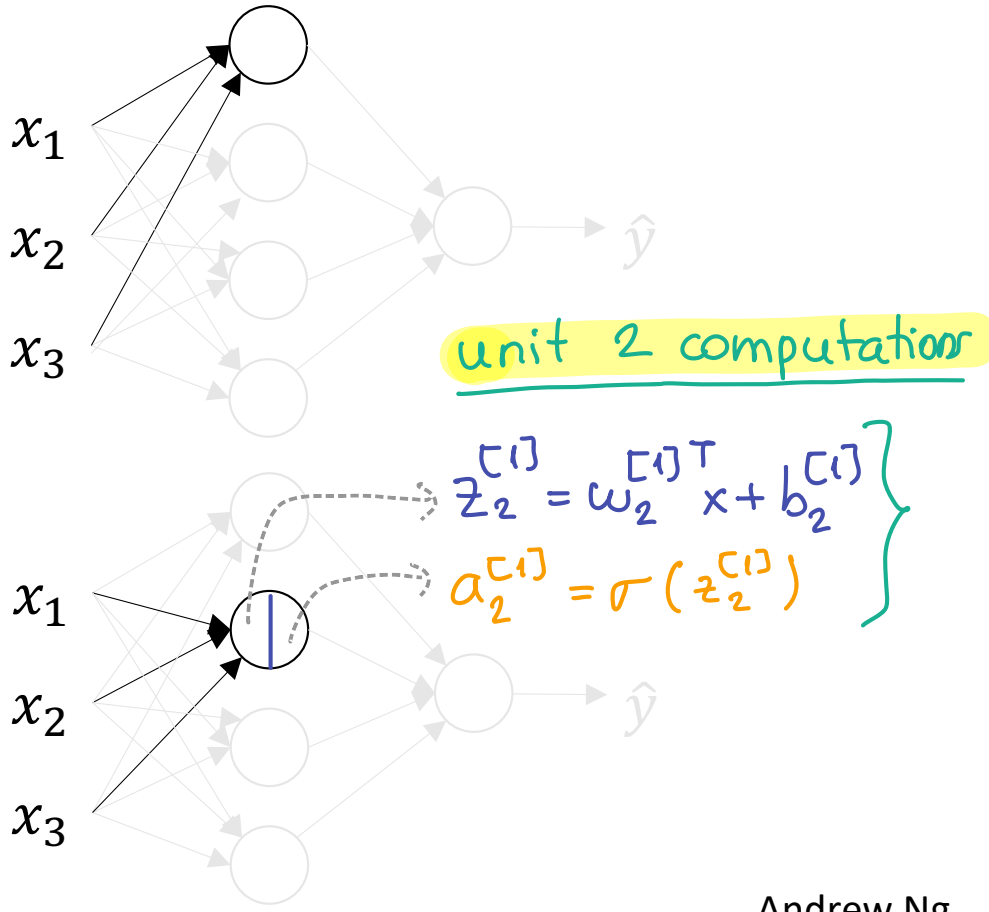
Andrew Ng

Neural Network Representation



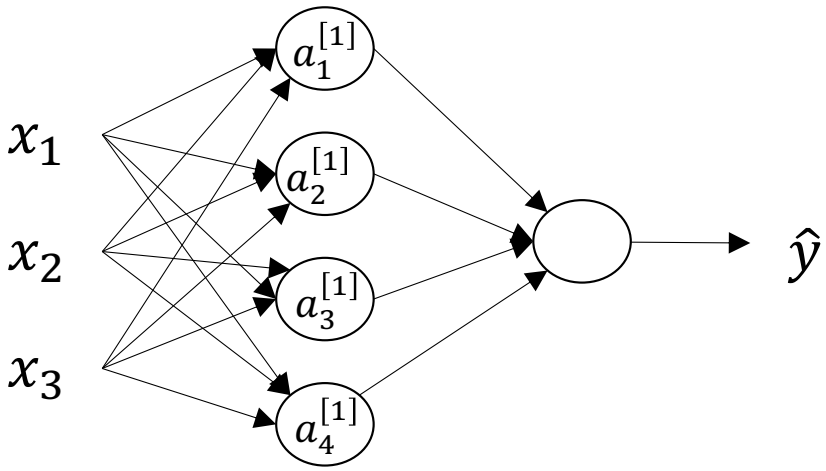
$$z = w^T x + b$$

$$a = \sigma(z)$$



Andrew Ng

Neural Network Representation



$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]},\ a_1^{[1]} = \sigma(z_1^{[1]})$$

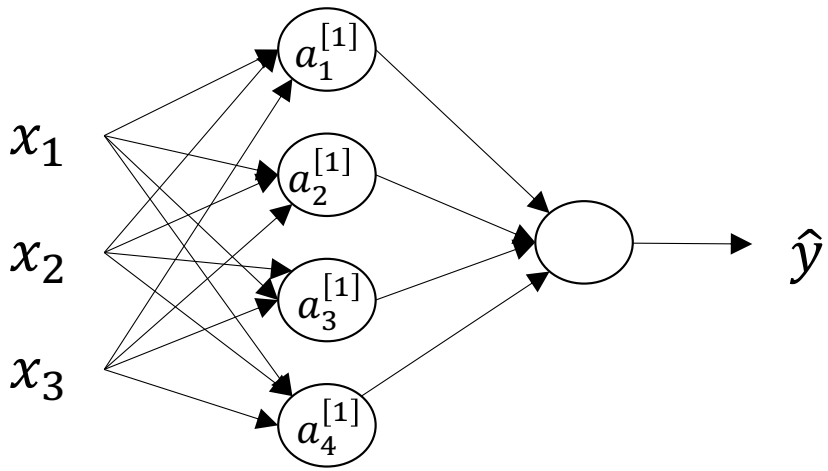
$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]},\ a_2^{[1]} = \sigma(z_2^{[1]})$$

$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]},\ a_3^{[1]} = \sigma(z_3^{[1]})$$

$$z_4^{[1]} = w_4^{[1]T} x + b_4^{[1]},\ a_4^{[1]} = \sigma(z_4^{[1]})$$

Goal: vectorize the above 4 equations.

Neural Network Representation



$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]}, \quad a_1^{[1]} = \sigma(z_1^{[1]})$$
$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]}, \quad a_2^{[1]} = \sigma(z_2^{[1]})$$
$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]}, \quad a_3^{[1]} = \sigma(z_3^{[1]})$$
$$z_4^{[1]} = w_4^{[1]T} x + b_4^{[1]}, \quad a_4^{[1]} = \sigma(z_4^{[1]})$$

put in a vector: $z^{[1]}$

Step 1: Put the weights in a matrix:

$$\begin{bmatrix} \text{---} & w_1^{[1]T} & \text{---} \\ \text{---} & w_2^{[1]T} & \text{---} \\ \text{---} & w_3^{[1]T} & \text{---} \\ \text{---} & w_4^{[1]T} & \text{---} \end{bmatrix} \quad (4,3)$$

- as if we have 4 logistic regression units

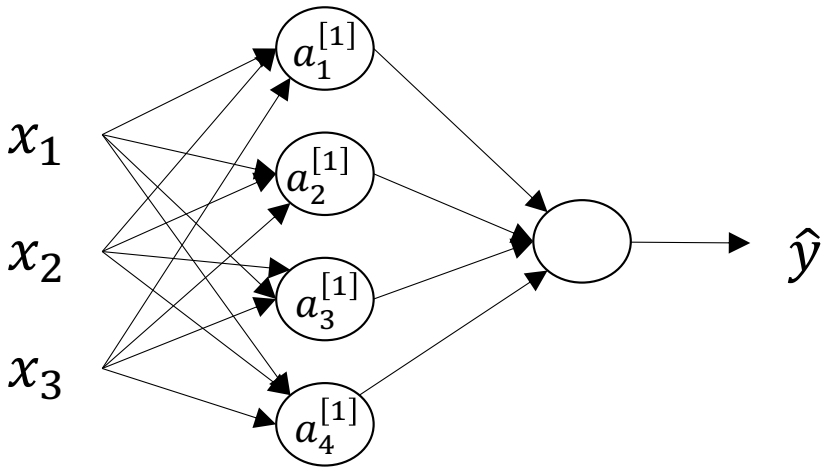
Andrew Ng

- Multiply the weight matrix and the input vector:

$$\underbrace{\begin{bmatrix} — & w_1^{[1]T} & — \\ — & w_2^{[1]T} & — \\ — & w_3^{[1]T} & — \\ — & w_4^{[1]T} & — \end{bmatrix}}_{W^{[1]}:\text{matrix}}_{(4,3)} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{x} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}}_{b^{[1]}:\text{vector}} = \begin{bmatrix} w_1^{[1]T}x + b_1^{[1]} \\ w_2^{[1]T}x + b_2^{[1]} \\ w_3^{[1]T}x + b_3^{[1]} \\ w_4^{[1]T}x + b_4^{[1]} \end{bmatrix} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = z^{[1]}$$

$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} = \sigma(z^{[1]})$$

Neural Network Representation learning



Given input x:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

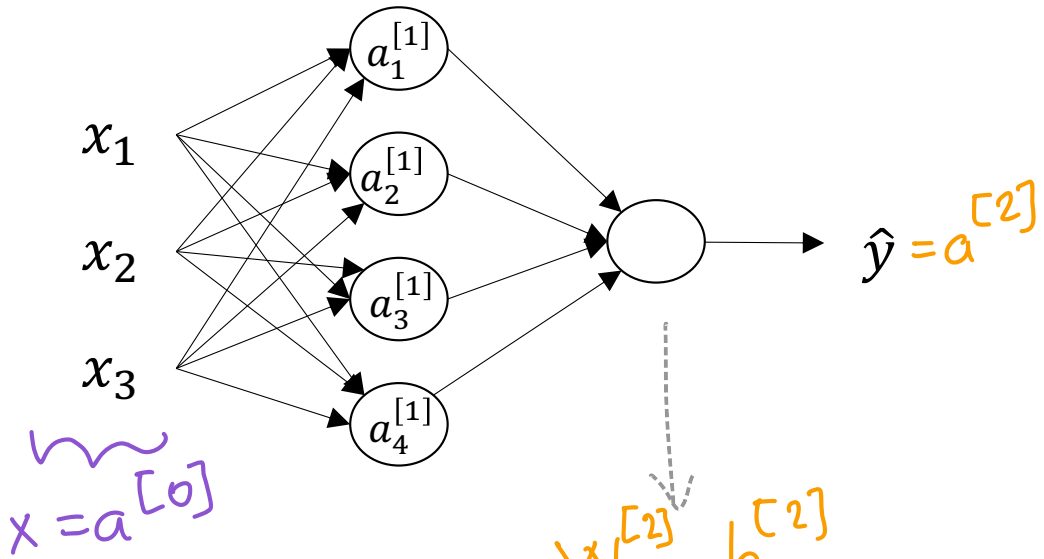
$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

Andrew Ng

Neural Network Representation learning



Given input x: $a^{[0]}$

$$z^{[1]} = W^{[1]}x + b^{[1]}$$
(4,1) (4,3)(3,1) (4,1) : dimens.

$$a^{[1]} = \sigma(z^{[1]})$$
(4,1) (4,1)

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$
(1,1) (1,4) (4,1) (1,1)

$$a^{[2]} = \sigma(z^{[2]})$$
(1,1) (1,1)

4-lines of code for a single example

Andrew Ng

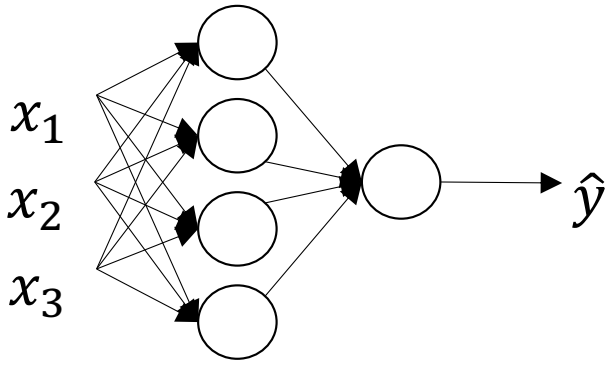


deeplearning.ai

One hidden layer Neural Network

Vectorizing across multiple examples

Vectorizing across multiple examples



$$z^{[1]} = W^{[1]}x + b^{[1]}$$
$$a^{[1]} = \sigma(z^{[1]})$$
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$
$$a^{[2]} = \sigma(z^{[2]})$$

$x \rightarrow a^{[2]} = \hat{y}$

$x^{(1)} \rightarrow a^{[2](1)} = \hat{y}^{(1)}$

$x^{(2)} \rightarrow a^{[2](2)} = \hat{y}^{(2)}$

$x^{(m)} \rightarrow a^{[2](m)} = \hat{y}^{(m)}$

$a^{[2](i)}$ → example i, layer 2

for $i = 1$ to m, # for m examples

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$
$$a^{[1](i)} = \sigma(z^{[1](i)})$$
$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$
$$a^{[2](i)} = \sigma(z^{[2](i)})$$

Andrew Ng

Goal: Vectorize to eliminate the for loop over m examples

Vectorizing across multiple examples

```
for i = 1 to m:
```

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

Vectorized version:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

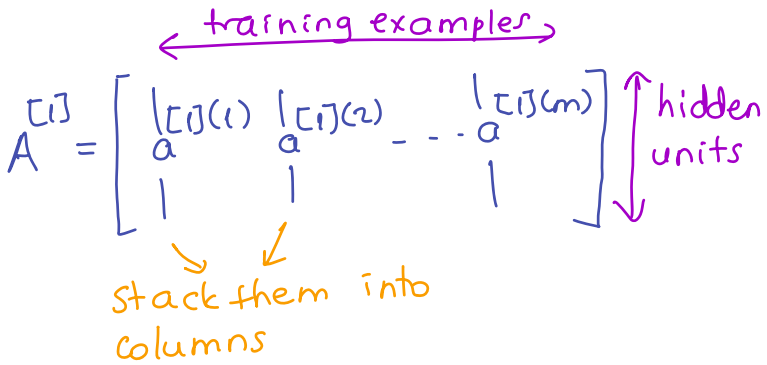
(n_x, m)

where

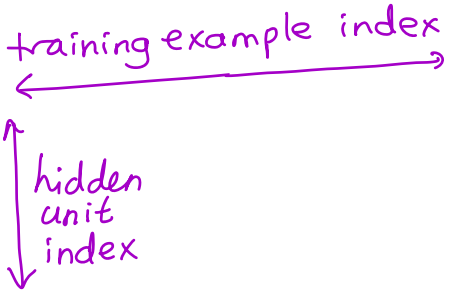
$$Z^{[1]} = \begin{bmatrix} | & | & & | \\ z^{[1](1)} & z^{[1](2)} & \cdots & z^{[1](m)} \\ | & | & & | \end{bmatrix}$$

↳ column vector

Andrew Ng



for X, Z and A:



- You know how to implement a NN with vectorization.



deeplearning.ai

One hidden layer Neural Network

Explanation for vectorized implementation

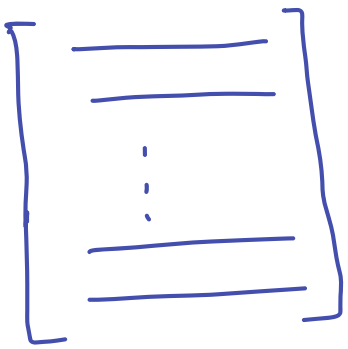
Justification for vectorized implementation

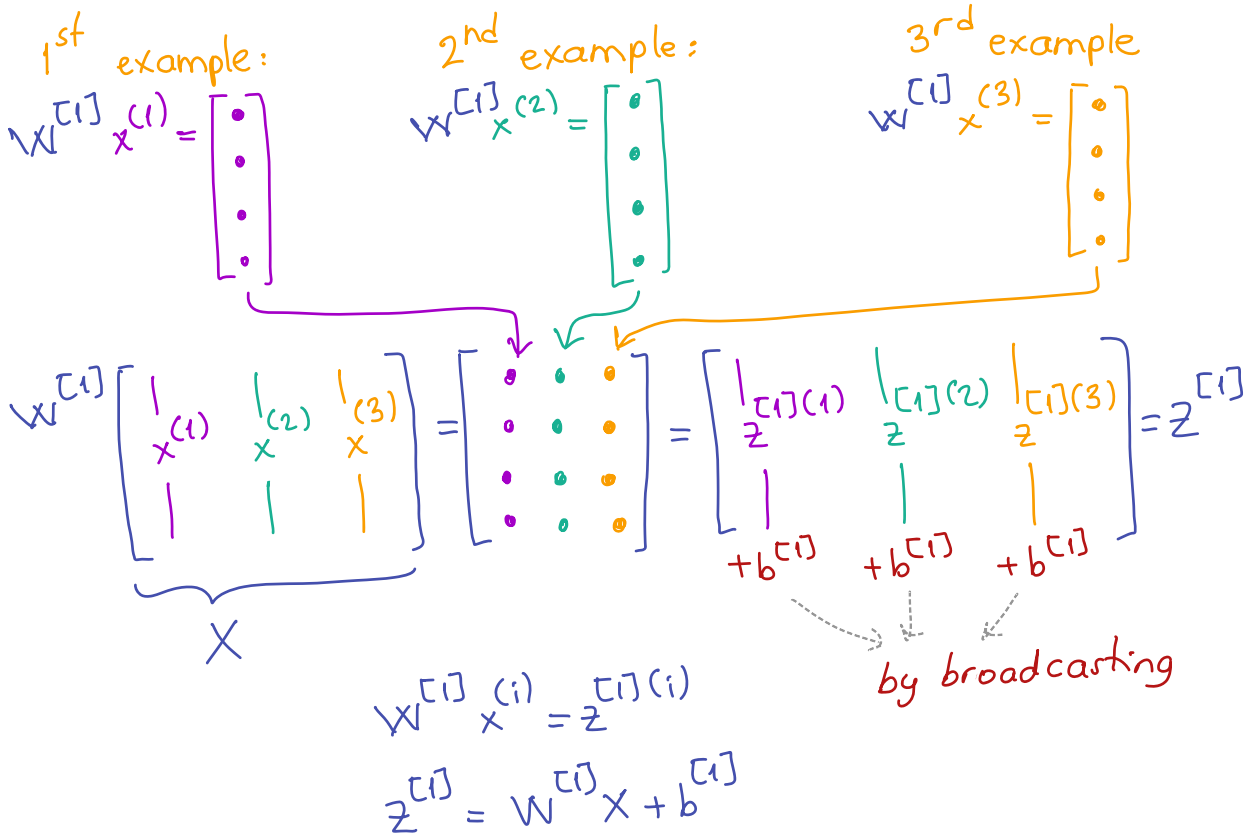
Let's go through forward propagation for a few examples:

$z^{[1](1)} = W^{[1]} x^{(1)} + b^{[1]}$, $z^{[1](2)} = W^{[1]} x^{(2)} + b^{[1]}$, $z^{[1](3)} = W^{[1]} x^{(3)} + b^{[1]}$

1st example 2nd example 3rd example

To simplify let $b^{[1]} = 0$

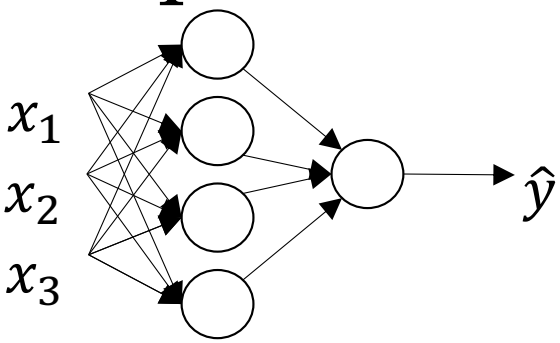
$W^{[1]}$ a matrix $=$ 



$$W^{[1]} x^{(i)} = z^{[1](i)}$$

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

Recap of vectorizing across multiple examples



$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

stack training examples

$$A^{[1]} = \begin{bmatrix} | & | & & | \\ a^{[1](1)} & a^{[1](2)} & \cdots & a^{[1](m)} \\ | & | & & | \end{bmatrix}$$

```
for i = 1 to m
```

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$
$$a^{[1](i)} = \sigma(z^{[1](i)})$$
$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$
$$a^{[2](i)} = \sigma(z^{[2](i)})$$

vectorize over all m samples

$A^{[0]} = X$

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$
$$A^{[1]} = \sigma(Z^{[1]})$$
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$
$$A^{[2]} = \sigma(Z^{[2]})$$

same

- Different layers are doing the same computation over and over.

Andrew Ng



deeplearning.ai

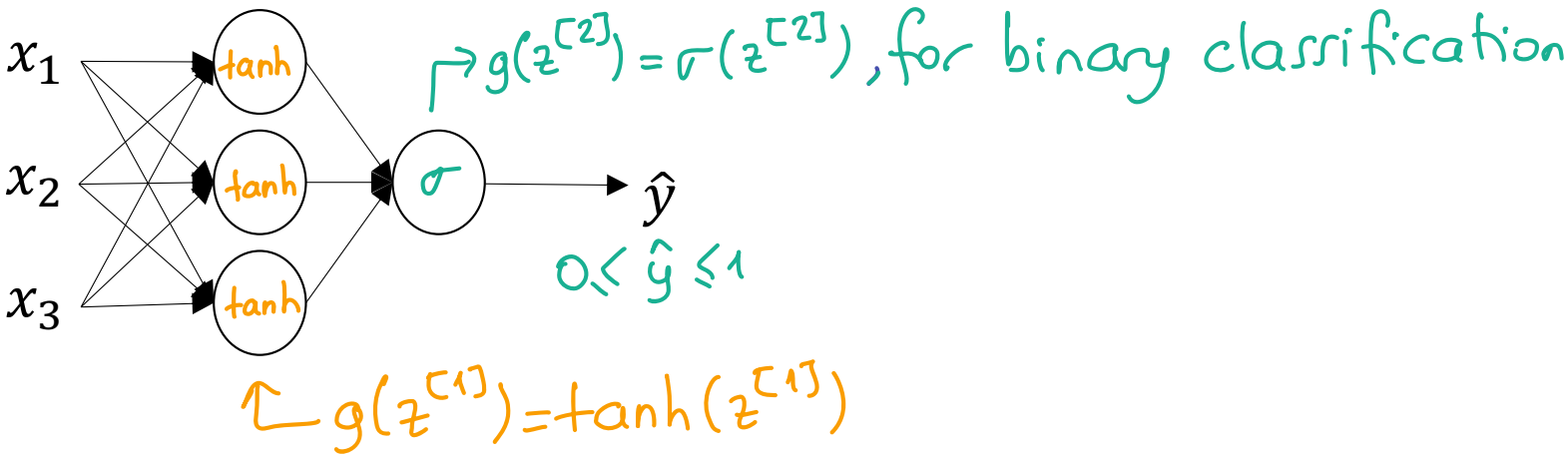
One hidden layer Neural Network

Activation functions

Activation Functions

- So far, we have been using the sigmoid activation function.
- Sometimes other activation functions may work better.

Activation functions



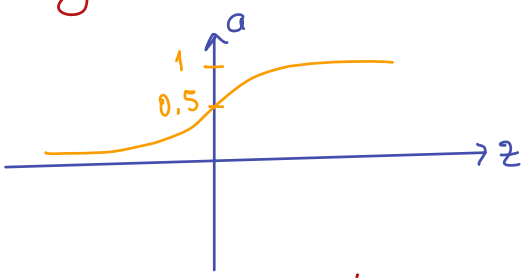
Given x:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$
$$a^{[1]} = \sigma(z^{[1]}) \rightarrow g^{[1]}(z^{[1]})$$
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$
$$a^{[2]} = \sigma(z^{[2]}) \rightarrow g^{[2]}(z^{[2]})$$

$g^{[1]}(z^{[1]})$: $g(\cdot)$ is another nonlinear activation function

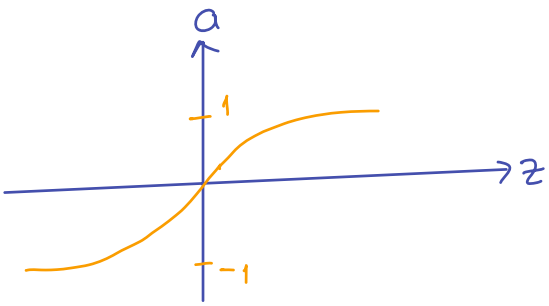
Andrew Ng

Sigmoid Function:



$$a = \frac{1}{1+e^{-z}} = \frac{e^{z}}{e^{z}+1} \in (0,1)$$

Tangent Hyperbolic Function: almost always works better than the sigmoid function.



$$a = \tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$

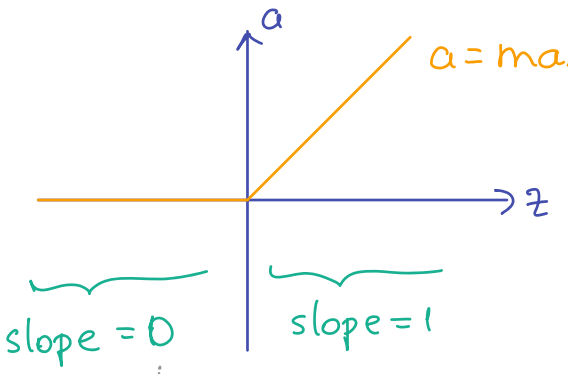
(shifted version of sigmoid function)

- Tanh is better than sigmoid function in the hidden layers since $\tanh \in [-1,1]$ and centered around zero, and activations will have approx. zero mean (i.e. centered). This makes learning for the next layer easier.
- we want $0 \leq \hat{y} \leq 1$ since it represents a probability. Hence, it is better to use the sigmoid activation in the last layer.

Disadvantage of Sigmoid and Tanh functions:

- When z becomes very large or very small, the sigmoid and tanh functions are almost flat, hence derivatives (slopes) are almost zero.
- This slows down gradient descent.
- One other popular activation function is ReLU.

ReLU (Rectified Linear Unit) Activation function

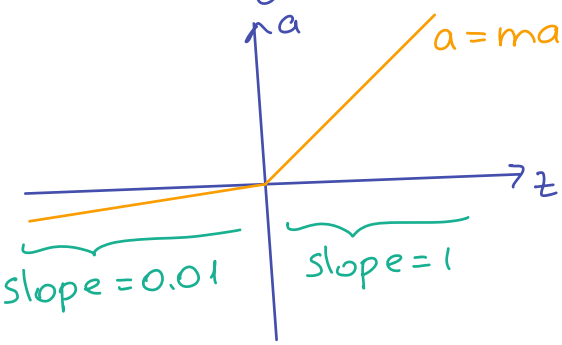


- Discontinuous at $z=0$ ⇒ not differentiable at $z=0$.
- In practice assume derivative is 0 or 1 at $z=0$.

disadvantage, but in practice works fine, since enough hidden units have $z>0$.

Leaky ReLU activation function

- Has a slight slope when z is negative

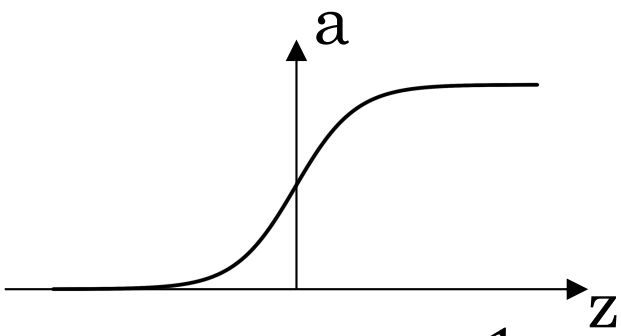


The leaky ReLU usually works better than ReLU in practice.

Rules of Thumb for Choosing Activation Functions

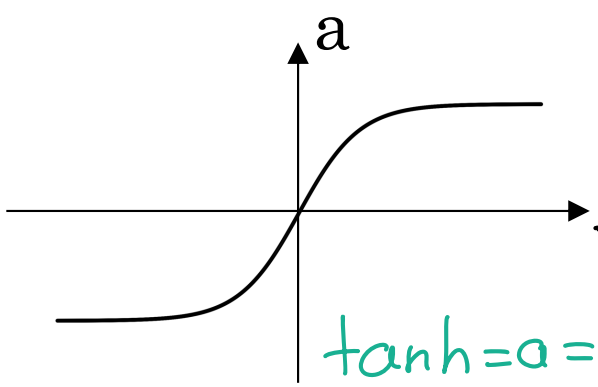
- If your output is between 0 and 1 (e.g. in binary classification), then sigmoid activation is a natural choice for output layer.
- For all other units ReLU or tanh maybe better.
- If you're not sure what to use for hidden layer, use ReLU. or leaky ReLU. (Or try them all and choose the best.
- Advantage of ReLU and leaky ReLU: for a lot of z values, the derivative is is very different from zero ⇒ network will learn much faster than with tanh and sigmoid.

Pros and cons of activation functions

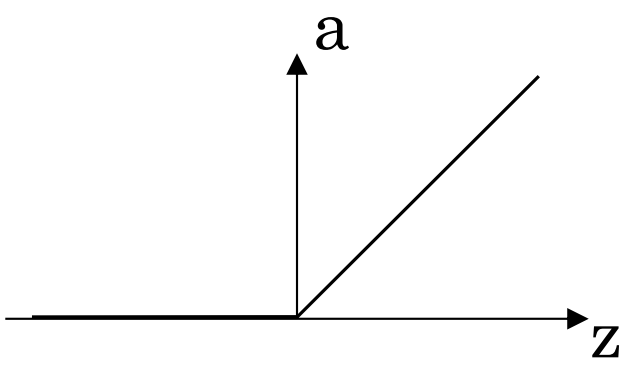


sigmoid: $a = \dfrac{1}{1+e^{-z}}$

never use except output layer of binary classification

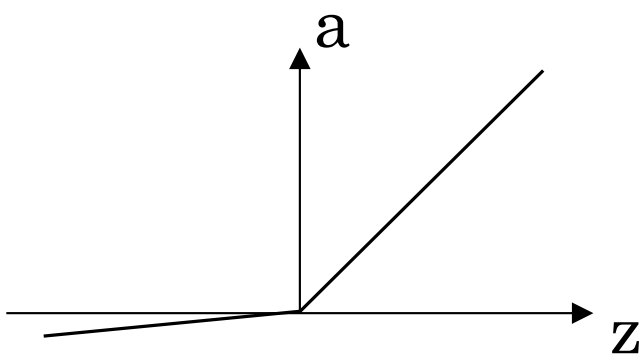


$\tanh = a = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$



most commonly used

ReLU = max(0, z)



Leaky ReLU a = max(0.01z, z)

Andrew Ng

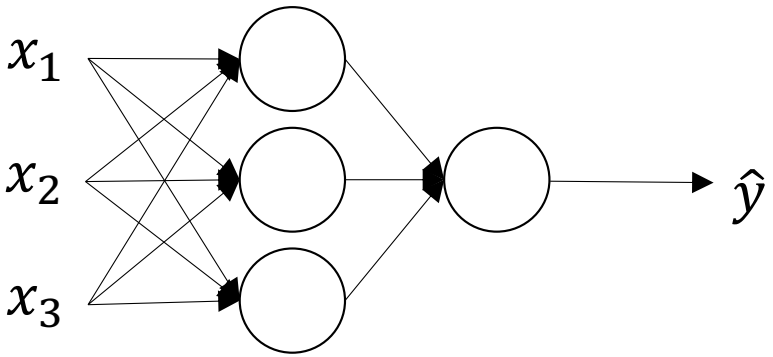


deeplearning.ai

One hidden layer Neural Network

Why do you need non-linear activation functions?

Activation function



- Why do we need non-linear activation functions?

Given x:

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]}) = z^{[1]}$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]}) = z^{[2]}$$

Let's use $g(z) = z$ instead "linear activation function" (identity)

Andrew Ng

If $g(z) = z$

$a^{[1]} = z^{[1]} = W^{[1]}x + b^{[1]}$

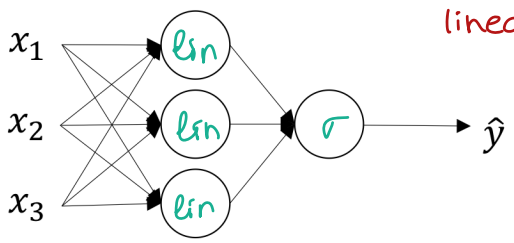
$a^{[2]} = z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$

$a^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}$

$= (W^{[2]}W^{[1]})x + W^{[2]}b^{[1]} + b^{[2]}$

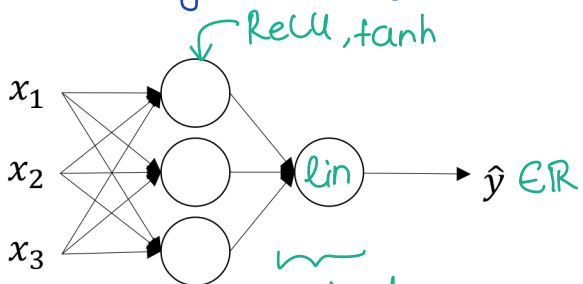
call W' call b'

$a^{[2]} = W'x + b' \Rightarrow$ The network outputs a linear function of the input (no matter how many layer it has) $\Rightarrow$ A linear hidden layer is useless. since composition of two linear functions is linear.



} similar to logistic regression

- One place where you may ure a linear actuation function is, when you're doing regression.



output layer may be linear.

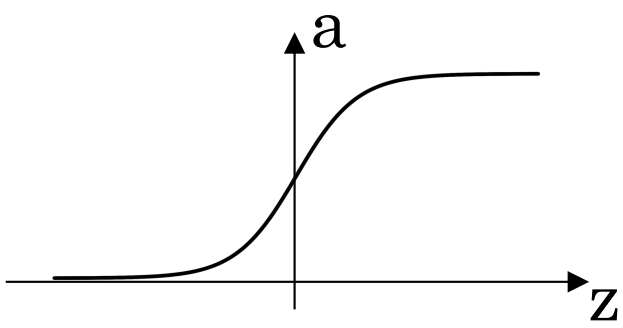


deeplearning.ai

One hidden layer Neural Network

Derivatives of activation functions

Sigmoid activation function



$$g(z) = \frac{1}{1+e^{-z}} \quad (1)$$

HW: show the derivation

$\frac{d}{dz} g(z)$ = slope of $g(z)$ at z

$$= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}}\right)$$

$$= g(z)\left(1-g(z)\right) \quad (2)$$

$$g'(z) = a(1-a)$$

Andrew Ng

check to see if the formula makes sense.

if z is large e.g. $z=10 \Rightarrow g(z) \approx 1$ (from (1))

$$\frac{d}{dz} g(z) \approx 1\,(1-1) \approx 0 \quad \text{(from (2))}$$

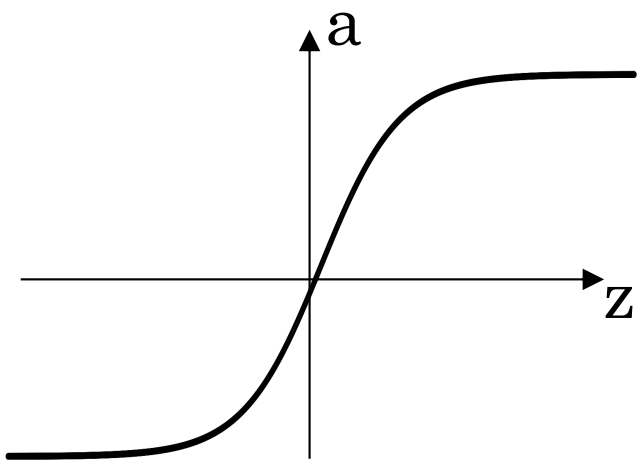
if z is small, e.g. $z=-10 \Rightarrow g(z) \approx 0$ (from (1))

$$\frac{d}{dz} g(z) \approx 0 \cdot (1-0) \approx 0 \quad \text{(from (2))}$$

if $z=0 \Rightarrow g(z) = \frac{1}{2}$

$$\frac{d}{dz} g(z) = \frac{1}{2}\left(1-\frac{1}{2}\right) = \frac{1}{4}$$

Tanh activation function



$$g(z) = \tanh(z) = a$$

$$= \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Show as HW

$g'(z) = \frac{d}{dz} g(z)$ = slope of $g(z)$ at a certain z value

$$g'(z) = 1 - (\tanh(z))^2 = 1 - a^2$$

Check the formula:

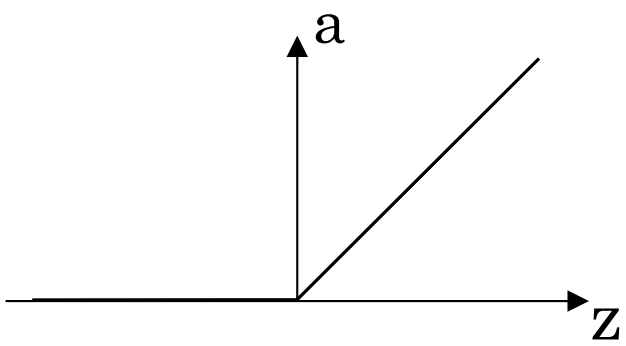
let $z = 10 \Rightarrow \tanh(z) \approx 1$, $g'(z) \approx 0$

let $z = -10 \Rightarrow \tanh(z) \approx -1$, $g'(z) = 1 - (-1)^2 \approx 0$

if $z = 0 \Rightarrow \tanh(z) = 0$, $g'(z) = 1$

Andrew Ng

ReLU and Leaky ReLU

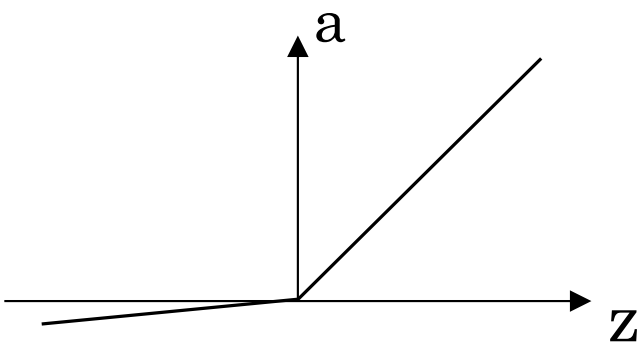


ReLU

$g(z) = \max(0, z)$

$g'(z) = \begin{cases} 0 & \text{, if } z < 0 \\ 1 & \text{, if } z \geq 0 \end{cases}$

derivative is undefined at $z = 0$, but we set it to 1 for practical implementation (subgradient)



Leaky ReLU

$g(z) = \max(0.01z, z)$

$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$

Andrew Ng



deeplearning.ai

One hidden layer Neural Network

Gradient descent for neural networks

Gradient descent for neural networks

- How to implement gradient descent for a NN with one hidden layer.

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$

$(n^{[1]}, n^{[0]})$ $(n^{[1]}, 1)$ $(n^{[2]}, n^{[1]})$ $(n^{[2]}, 1)$

$n_x = n^{[0]}$: # inputs

$n^{[1]}$: # hidden units

$n^{[2]} = 1$: # outputs

Cost function: $J(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}, y)$

$\hat{y} = a^{[2]}$

Andrew Ng

Gradient Descent:

Repeat {

Compute predictions: $\hat{y}^{(i)}$, $i = 1, \ldots, m$)

Compute derivatives: $dW^{[1]} = \frac{dJ}{dW^{[1]}}$, $db^{[1]} = \frac{dJ}{db^{[1]}}$, $dW^{[2]}$, $db^{[2]}$

GD update: $W^{[1]} := W^{[1]} - \alpha dW^{[1]}$

$b^{[1]} := b^{[1]} - \alpha db^{[1]}$

$W^{[2]} := W^{[2]} - \alpha dW^{[2]}$

$b^{[2]} := b^{[2]} - \alpha db^{[2]}$

}

Formulas for computing derivatives

Forward Propagation:

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(Z^{[1]})$$

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(Z^{[2]})$$

$$= \sigma(Z^{[2]})$$

for binary classification

Backpropagation:

$$dZ^{[2]} = A^{[2]} - Y \qquad (Y = [y^{(1)}\ y^{(2)}\ \cdots\ y^{(m)}])$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis}=1, \text{keepdims}=\text{True})$$

gives $(n^{[2]}, 1)$; sum horizontally; prevents $(n,)$

$$dZ^{[1]} = \underbrace{W^{[2]T} dZ^{[2]}}_{(n^{[1]}, m)} * \underbrace{g^{[1]\prime}(Z^{[1]})}_{(n^{[1]}, m)}$$

elementwise product

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$\underbrace{db^{[1]}}_{(n^{[1]}, 1)} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis}=1, \text{keepdims}=\text{True})$$

prevents $(n^{[1]}, 1)$

Andrew Ng



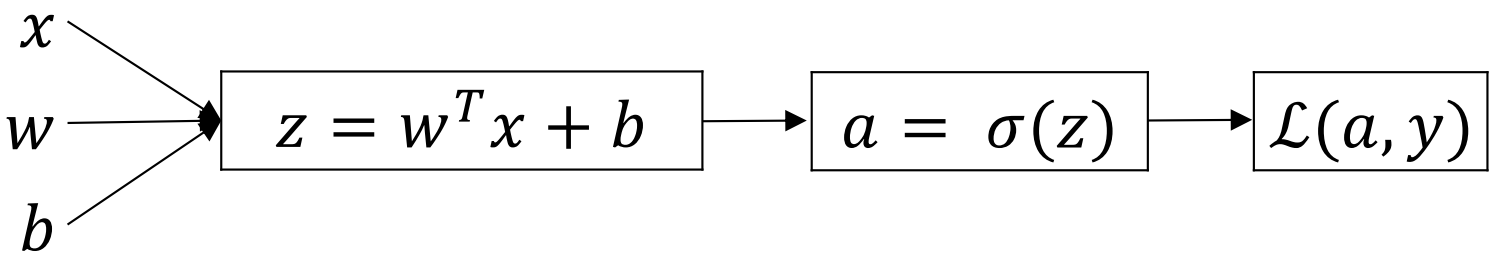
deeplearning.ai

One hidden layer Neural Network

Backpropagation intuition (Optional)

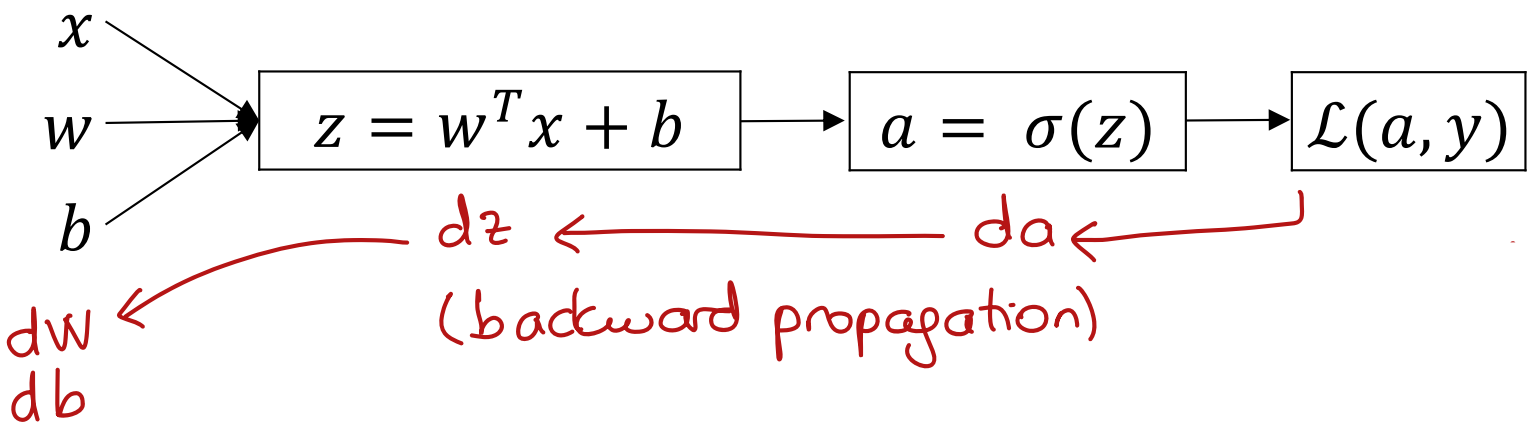
Computing gradients

Logistic regression



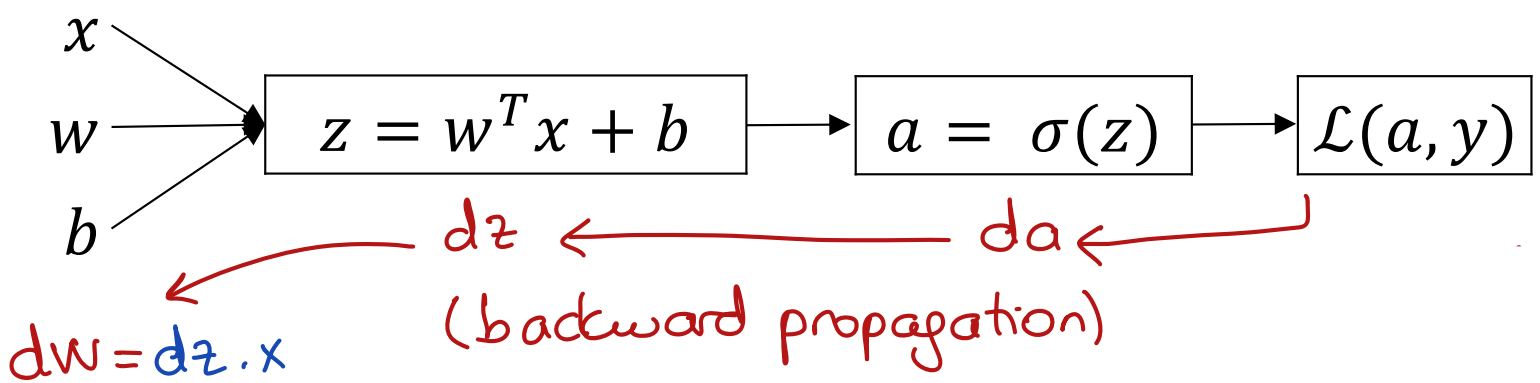
Computing gradients

Logistic regression (Computation Graph)



Computing gradients

Logistic regression (Computation Graph)



$$\mathcal{L}(a,y) = -y \log a - (1-y)\log(1-a)$$

$$da = \frac{\partial}{\partial a}\mathcal{L}(a,y) = -\frac{y}{a} + \frac{1-y}{1-a}$$

Andrew Ng

$$dz = \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{da}{dz}$$ (chain rule)

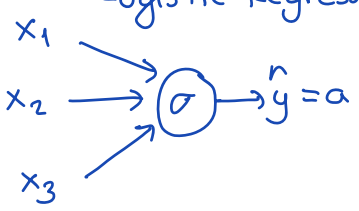
$a = \sigma(z) = g(z)$

$$dz = da \cdot g'(z)$$

after a few steps

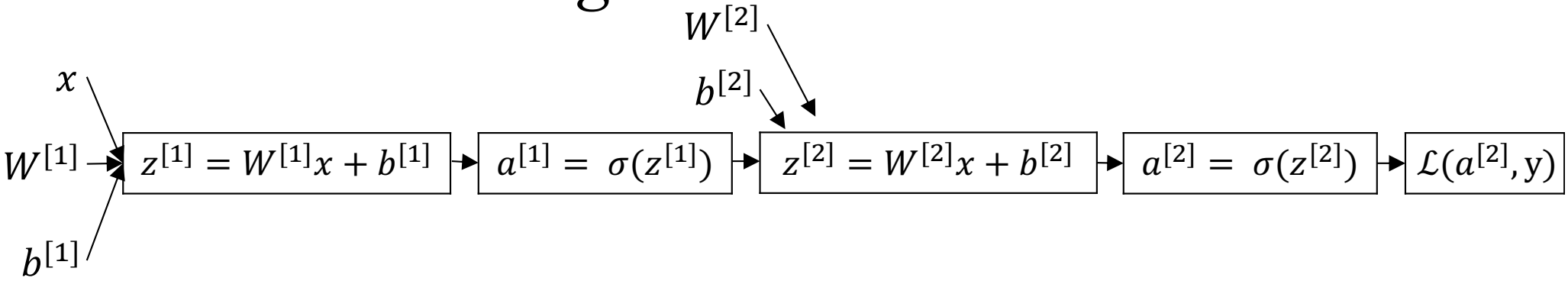
$$dz = a - y$$

Logistic Regression



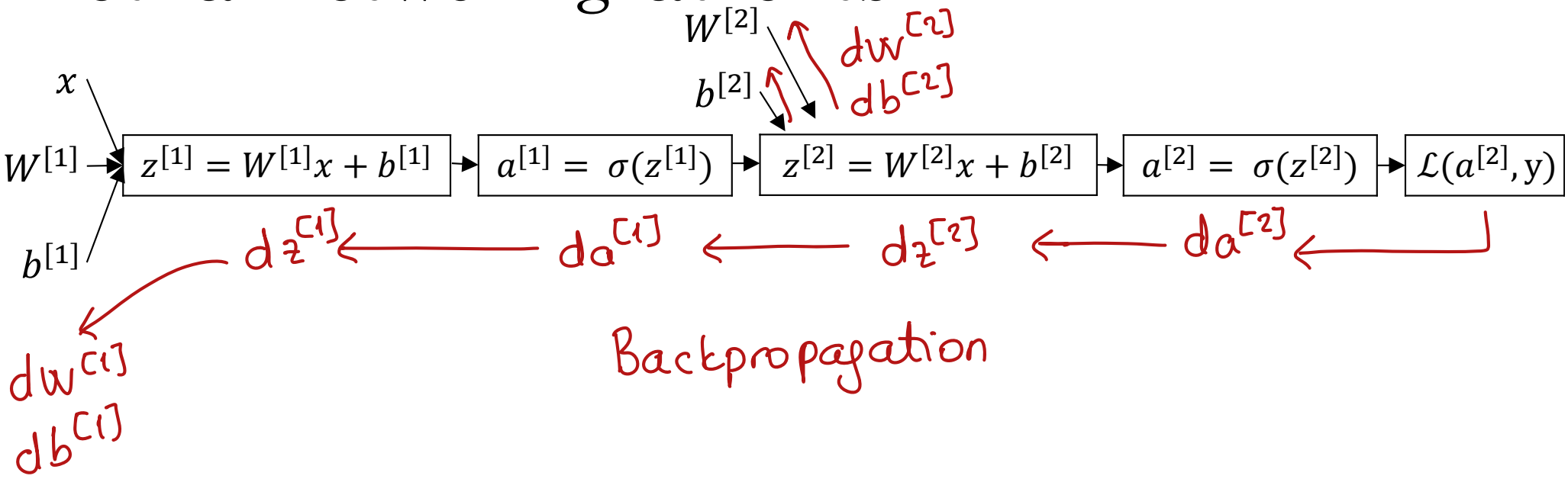
Two-Layer Neural Network

Neural network gradients



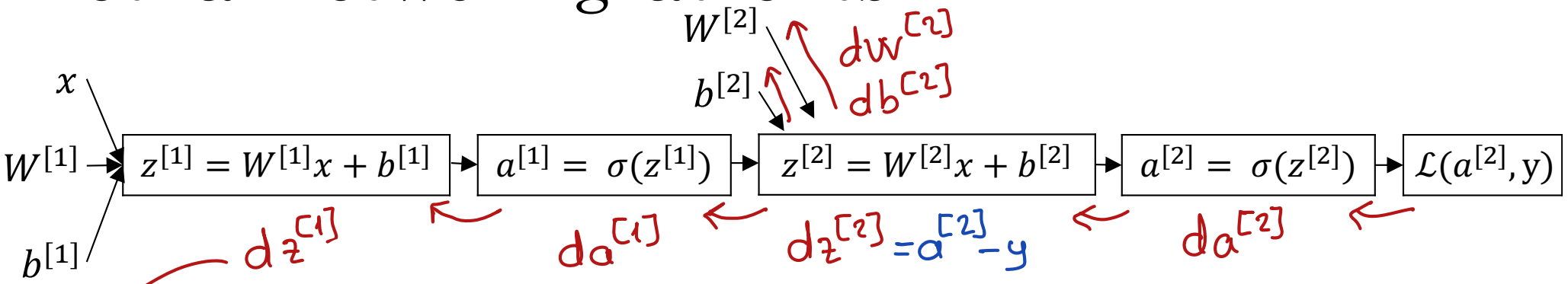
Two-Layer Neural Network

Neural network gradients



Two-Layer Neural Network

Neural network gradients



Andrew Ng

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

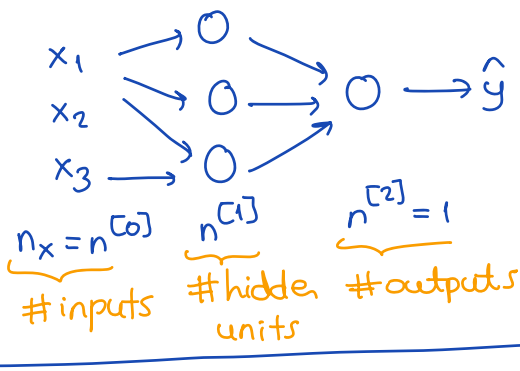
↳ elementwise product

Check the dimensions:

$W^{[2]} : (n^{[2]}, n^{[1]})$

$z^{[2]}, dz^{[2]} : (n^{[2]}, 1) \rightarrow (1,1)$ binary classific.

$z^{[1]}, dz^{[1]} : (n^{[1]}, 1)$



$$\underbrace{dz^{[1]}}_{(n^{[1]},1)} = \underbrace{W^{[2]T}}_{(n^{[1]},n^{[2]})} \underbrace{dz^{[2]}}_{(n^{[2]},1)} * \underbrace{g^{[1]'}(z^{[1]})}_{(n^{[1]},1)}$$

same

$(n^{[1]},1) = (n^{[1]},1) * (n^{[1]},1)$ ↳ dimensions match

- When implementing backpropagation, make sure that the dimensions match, which will prevent many errors.

$dW^{[1]} = dz^{[1]} \cdot x^T$ ↳ $a^{[0]T}$

$db^{[1]} = dz^{[1]}$

Summary of gradient descent

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]\prime}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

For a single training example

For all examples:

Vectorized implementation:

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(Z^{[1]})$$

Forward Propagation

to vectorize we stacked examples into columns

$$Z^{[1]} = \begin{bmatrix} | & | & & | \\ z^{[1](1)} & z^{[1](2)} & \cdots & z^{[1](m)} \\ | & | & & | \end{bmatrix}$$

capital

Andrew Ng

Summary of gradient descent

For a single example x:

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]\prime}(z^{[1]})$$

$(n^{[1]}, 1)$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

Vectorized implementation:

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$J(\ldots) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}, y)$$

$$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1, keepdims = True)$$

elementwise multiplication

$$dZ^{[1]} = \underbrace{W^{[2]T} dZ^{[2]}}_{(n^{[1]}, m)} * \underbrace{g^{[1]\prime}(Z^{[1]})}_{(n^{[1]}, m)}$$

$(n^{[1]}, m)$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$$

Andrew Ng

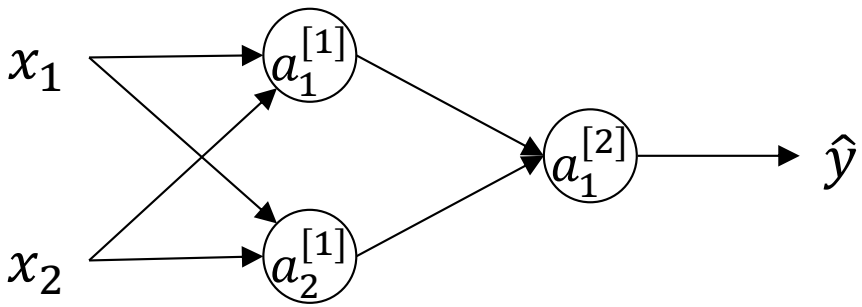


deeplearning.ai

One hidden layer Neural Network

Random Initialization

What happens if you initialize weights to zero?



$n^{[0]}=2$ $n^{[1]}=2$

Let's initialize all the weights to zero:

$$W^{[1]}=\begin{bmatrix}0 & 0\\ 0 & 0\end{bmatrix} \qquad b^{[1]}=\begin{bmatrix}0\\ 0\end{bmatrix} \qquad W^{[2]}=\begin{bmatrix}0 & 0\end{bmatrix}$$

a problem

this is OK in practice

What's the problem? $a_1^{[1]}=a_2^{[1]}$ → both hidden units will compute the same activations

Andrew Ng

Then, during backpropagation:

$$dz_1^{[1]} = dz_2^{[1]}$$

$\Rightarrow$ Both hidden units are symmetric after backprop.

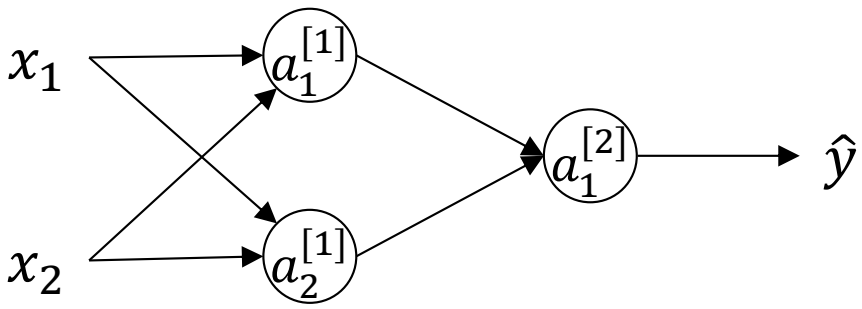
since they're computing the same thing:

$$dW = \begin{bmatrix} u & v \\ u & v \end{bmatrix}, \quad W^{[1]} := W^{[1]} - \alpha\, dW, \quad W^{[1]} = \begin{bmatrix} \text{——} \\ \text{——} \end{bmatrix} \begin{matrix} \rightarrow \text{same} \\ \rightarrow \text{rows} \end{matrix}$$

$\Rightarrow$ No point of using more than 1 hidden unit

SOLUTION:

Random initialization



$W^{[1]}$ = np.random.randn((2,2)) * 0.01 #initialize to very small random values

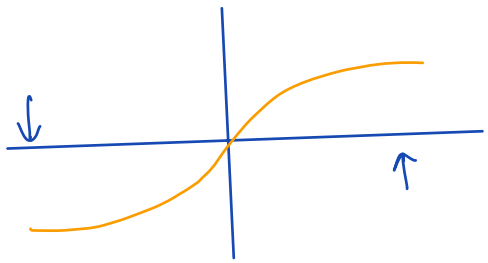
$b^{[1]}$ = np.zero((2,1)) # we don't have the "symmetry breaking problem for b, if W is initialized randomly.

$W^{[2]}$ = np.random.randn((1,2)) * 0.01

$b^{[2]}$ = 0

Andrew Ng

- Why do we initialize to very small random numbers? (i.e. why 0.01 but not 100?)



$$z^{[1]} = W^{[1]} x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

- For tanh or sigmoid function, when the weights are large, we may end up at the flat regions of the activation function, where the slope will be small.
  - Therefore gradient descent will be slow.
  - Learning will be slow.

- If you don't have sigmoid or tanh activation functions in the network, this may not be an issue.
- But for binary classification, we have a sigmoid function in the final layer.
- For very deep networks, other constants than (0.01) may work better.